

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки
кафедра «Радіоелектронні пристрої та системи»



Звіт з лабораторної роботи №8
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи ІХ-21
Пасько Максим Павлович

Прийняла:
к.т.н., доц. каф. ПАСІ
Плесканка М.В.

Львів 2026р.

Лабораторна робота №8 Реалізація клієнт-серверної взаємодії з використанням протоколу MQTT, автоматизація передачі даних через API.

Мета: Розробити клієнт-серверну архітектуру з використання MQTT протоколу, реалізовувати класи для автоматизації роботи з API.

Хід виконання роботи

Завдання до лабораторної роботи

1. Розробити MQTT клієнт MQTTClient, який дозволяє публікувати повідомлення на задану тему на реальному MQTT брокері.

Вимоги до завдання:

- **Створити клас MQTTClient, який містить методи:**
 - connect(): для підключення до MQTT брокера.
 - publish(topic, message): для публікації повідомлення на задану тему.
 - disconnect(): для відключення від брокера.
- **Використовувати реальний MQTT брокер для тестування:**
 - Наприклад, можна використовувати сервіси:
 1. **CloudMQTT** — безкоштовний MQTT брокер з обмеженням на кількість підключень.
 2. **HiveMQ** — безкоштовний сервіс для тестування MQTT.
 - Зареєструйтесь на одному з цих сервісів та отримайте необхідні дані для підключення до брокера (адреса сервера, порт, логін, пароль).
 - Можна використати сервіси без реєстрації, наприклад **broker.hivemq.com**
- **Інструкція по підключенню до MQTT брокера:**
 - Зареєструйтесь на обраному MQTT сервісі (наприклад, CloudMQTT або HiveMQ).
 - Отримайте наступні дані:
 1. **Адреса сервера** (наприклад, mqtt://mqtt.eclipse.org).
 2. **Порт** для підключення (наприклад, 1883 для незахищеного з'єднання або 8883 для захищеного через TLS/SSL).

3. **Тема** для публікації (наприклад, home/temperature).

- Використовуйте ці дані для налаштування підключення в класі MQTTClient.

1.1. Інтеграція всіх компонентів у єдину систему:

- Створіть програму, яка:
 - Отримує дані через REST API.
 - Передає отримані дані через WebSocket.
 - Публікує ці дані через MQTT.
- Вибрані API сервіси повинні працювати разом для передачі даних у режимі реального часу.

```
1  import requests
2  import json
3  import time
4  import websocket
5  import paho.mqtt.client as mqtt
6
7  # MQTT CLIENT
8  class MQTTClient: 1 usage
9
10     def __init__(self, broker="broker.hivemq.com", port=1883):
11         self.broker = broker
12         self.port = port
13         self.client = mqtt.Client()
14
15     def connect(self): 1 usage
16         print("Підключення до MQTT брокера...")
17         self.client.connect(self.broker, self.port)
18         print("Підключено до MQTT")
19
20     def publish(self, topic, message): 1 usage
21         self.client.publish(topic, message)
22         print(f"MQTT повідомлення відправлено в тему {topic}: {message}")
23
24     def disconnect(self):
25         self.client.disconnect()
26         print("Відключено від MQTT")
```

```

27
28 # REST API
29 def get_data_from_api(): 1 usage
30     url = "https://jsonplaceholder.typicode.com/posts/1"
31
32     response = requests.get(url)
33
34     if response.status_code == 200:
35         data = response.json()
36         print("Дані отримані з REST API:")
37         print(data)
38         return data
39     else:
40         print("Помилка отримання даних")
41         return None
42
43 # WEBSOCKET
44 def send_via_websocket(message): 1 usage
45
46     ws_url = "wss://ws.ifelse.io"
47
48     ws = websocket.create_connection(ws_url)
49
50     print("Підключено до WebSocket")
51
52     ws.send(json.dumps(message))
53     print("Дані відправлені через WebSocket")
54
55     result = ws.recv()
56     print("Відповідь WebSocket:", result)
57
58     ws.close()

```

```

59
60 # MAIN PROGRAM
61 def main(): 1 usage
62
63     topic = "home/temperature"
64
65     mqtt_client = MQTTClient()
66
67     mqtt_client.connect()
68
69     while True:
70
71         # 1. Отримуємо дані з API
72         data = get_data_from_api()
73
74         if data:
75
76             message = json.dumps(data)
77
78             # 2. Передаємо через WebSocket
79             send_via_websocket(data)
80
81             # 3. Публікуємо через MQTT
82             mqtt_client.publish(topic, message)
83
84             time.sleep(5)
85
86
87 if __name__ == "__main__":
88     main()

```

Виведе:

Підключення до MQTT брокера... #має затримка підключень

Підключено до MQTT

Дані отримані з REST API: #після цих вже повторюватись до безліч

```
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto'}
```

Підключено до WebSocket

Дані відправлені через WebSocket

Відповідь WebSocket: Request served by 6e82931b755587

```
MQTT повідомлення відправлено в тему home/temperature: {"userId": 1, "id": 1, "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"}
```

Дані отримані з REST API:

```
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto'}
```

Підключено до WebSocket

Дані відправлені через WebSocket

Відповідь WebSocket: Request served by 6e82931b755587

```
MQTT повідомлення відправлено в тему home/temperature: {"userId": 1, "id": 1, "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"}
```

Контрольні питання до лабораторної роботи

1. Що таке MQTT, і які його основні компоненти?

Це легкий протокол, що використовується для передачі даних в системах IoT (Internet of Things). Він розроблений для ефективної комунікації між пристроями з обмеженими ресурсами, забезпечуючи швидку та надійну передачу повідомлень.

Протокол базується на моделі публікації та підписки (Pub/Sub), де пристрої підписуються на теми і отримують повідомлення лише по цих темах.

2. Як працює механізм публікації та підписки в MQTT?

MQTT використовує модель publish/subscribe (публікація-підписка), яка дозволяє передавати повідомлення між пристроями через брокер.

Основні елементи MQTT:

Broker (брокер) – сервер, який приймає повідомлення і розсилає їх підписникам.

Publisher (видавець) – клієнт, який відправляє повідомлення.

Subscriber (підписник) – клієнт, який отримує повідомлення.

Topic (тема) – канал, через який передаються повідомлення.

Принцип роботи:

Клієнт підключається до MQTT-брокера.

Підписник підписується на певну тему (topic).

Видавець публікує повідомлення у цю тему.

Брокер автоматично передає повідомлення всім підписникам цієї теми.

3. Як інтегрувати REST API, WebSocket і MQTT у єдину систему для передачі даних у реальному часі?

MQTT використовується для передавання повідомлень між пристроями та сервером. Сервер отримує ці дані, може зберігати їх у базі даних і передавати клієнтам через WebSocket у реальному часі. REST API використовується для отримання даних, керування системою та виконання запитів від клієнтів. Така інтеграція дозволяє забезпечити швидку передачу даних і зручний доступ до них.